Lila Product Engineer Written Test

At Lila, we believe the role of a Product Engineer is evolving beyond its traditional scope. The convergence of PC/console and mobile gaming, the emergence of AI, and other global trends are creating a new environment that will fundamentally reshape how we build and ship exceptional gaming products.

We are looking for highly driven, intelligent, and creative problem-solvers who are passionate about delivering unique and compelling game experiences to players.
Our ambition isn't just to compete in India; we aim to challenge the best studios in the world with products that are proudly "Made in India."

We work on highly challenging and ambitious projects. If you are looking for a 9 to 5 job, this role is not right for you.

However, if you are ready for a massive challenge and willing to give it your all…

<< proceed forward >>

INSTRUCTIONS

1. We are looking for clear, concise, and well-thought-out responses
2. Use of AI to write any deliverables for this test is permitted
3. Deadline: 5 days from receiving this assignment.
4. Good luck!!!

Assignment: Player Journey Visualization Tool

Background

At LILA Games, our Level Design team needs a way to understand how players actually navigate our maps. Right now, they have raw telemetry data but no easy way to see what's happening visually — where players move, where fights break out, where people die to the storm, and which areas of the map get ignored.

Your job is to build a tool that turns this raw data into something a Level Designer can open in their browser and actually use.

What You're Getting

A zip file (`player_data.zip`) containing:

- **5 days of production gameplay data** from LILA BLACK (an extraction shooter)
- **Minimap images** for each of the 3 maps in the game
- **A README** explaining the data format, schema, and coordinate system

Read the README carefully before starting. Everything you need to understand the data is in there.

The Task

Build and deploy a web-based visualization tool that lets a Level Designer explore player behavior on our maps.

Requirements

Core (must have):

- Load and parse the provided parquet data
- Display player journeys on the correct minimap, with world coordinates properly mapped to the minimap image
- Distinguish between human players and bots visually
- Show different event types (kills, deaths, loot, storm deaths) as distinct markers
- Allow filtering by map, date, and/or match
- Timeline/playback to watch a match unfold over time
- Heatmap overlays showing kill zones, death zones, or high-traffic areas
- The tool must be hosted and accessible via a shareable link

What We're NOT Prescribing

- **Tech stack:** Use whatever you're comfortable with — React, Streamlit, plain HTML/JS, anything. Pick what lets you move fast and deliver something polished.
- **Design:** No mockups provided. Figure out what the right UX is for this kind of tool. Think about who's using it (Level Designers, not data scientists) and design accordingly.
- **Architecture:** You decide how to structure the data pipeline, frontend, and hosting. There are many valid approaches.

Guidelines

- **Use AI tools.** We expect you to use Claude, Cursor, ChatGPT, Copilot, or whatever helps you move faster. This is not a test of whether you can write code from scratch — it's a test of whether you can use the right tools to ship something that works.
- **Attention to detail matters.** The data has nuances (coordinate mapping, bytes encoding, bot detection, timestamps). Getting these right is part of the test.
- **Quality over quantity.** A polished tool with 4 well-executed features beats a messy tool with 10 half-working ones.
- **Document your assumptions.** If something is unclear, make a reasonable assumption and note it. Don't get blocked.
- **Timeline: 5 days.** We expect this to take roughly 10-15 hours of focused work.

What We're Evaluating

Area	**What we're looking for**
System design	Did you make sensible architecture and tech stack decisions? Does the data pipeline make sense?
Attention to detail	Are coordinates mapped correctly? Are events rendered accurately? Did you handle edge cases in the data?
End-to-end execution	Does the tool actually work? Is it hosted? Can we open it and use it without your help?
Product thinking	Did you build something a Level Designer would actually find useful? Are the right things filterable/interactive?
Code quality	Is the code organized, readable, and reasonably structured?
Communication	Does your architecture doc clearly explain your decisions? Can you articulate trade-offs?

What to Submit

Send us a single github repo link with everything we need to evaluate should be inside that repo .

- **Your code and a working deployment**
  - The repo should contain all source code for your tool
  - Include a deployed URL where we can use it (Vercel, Netlify, Railway, whatever works)
  - A README that tells tech stack,setup steps, env vars, etc.
- **An architecture doc (keep it to one page)**
  - Put this in an ARCHITECTURE.md file. We want to see:
  - What you built with and why you picked it
  - How data flows from the parquet files to what shows up on screen
  - How you mapped game coordinates to the minimap — this is the tricky part, so walk us through your approach
  - Any assumptions you made where the data was ambiguous — just tell us what you ran into and how you handled it
  - Major tradeoffs — what did you consider and what did you decide? A simple table works fine here
- **Three things you learned about the game using the tool you made**

  Create an INSIGHTS.md file. For each insight:
  - What caught your eye in the data
  - Back it up — point to a pattern, a stat, something concrete
  - Can you draw something actionable with this insight ? If yes , what metrics will get affected and what are the actionable items?
  - Why a level designer should care about it

Note : **Only github repo links with all files will be accepted , doc links or drive links will not be accepted**

Quick checklist before you submit

- ❖ Tool is live at the hosted URL
- ❖ Player paths render correctly on the minimap
- ❖ Can tell humans apart from bots visually

- Kill, death, loot, and storm events are marked
- Filtering by map/date/match works
- Timeline or playback shows match progression
- Heatmaps show kill zones, death zones, and traffic
- Architecture doc covers coordinate mapping approach
- Three insights with supporting evidence
- Walkthrough covers all major features

Good luck!!